

Concurrency and Distributed Systems

Week 4 – Part 3: Visibility vs Atomicity and Broken Double-Checked Locking

Dr. Mohamad Aoude

Faculty of Engineering

May 11, 2026

Part 3 Roadmap

- 1 From Visibility to Atomicity
- 2 The Lost Update Problem
- 3 Why volatile Is Not Enough
- 4 Connecting Back to Week 3
- 5 Broken Double-Checked Locking
- 6 Fixing DCL with volatile
- 7 Safer Alternatives and Lab Preparation

Where We Are in Week 4

Part 1

We learned why stale reads and visibility problems occur.

Part 2

We learned that `volatile` provides:

- visibility
- ordering guarantees
- happens-before through volatile communication

Part 3 challenge

A program can still be wrong even when visibility is correct.

Main question

Learning Objectives

By the end of this lecture, students should be able to:

- distinguish sharply between **visibility** and **atomicity**
- explain why `counter++` is not atomic
- analyze the lost update problem
- understand why locks from Week 3 are still relevant
- explain why broken double-checked locking was historically unsafe
- explain why `volatile` is required for correct DCL in modern Java
- identify safer alternatives to DCL

Visibility vs Atomicity

Visibility

Can one thread see another thread's update?

Examples:

- stop flag
- ready flag
- state publication

Atomicity

Does a multi-step operation behave like one indivisible action?

Examples:

- increment
- check-then-act
- initialize once

Critical distinction

A value can be perfectly visible and still be updated incorrectly.

The Trap: A Volatile Counter

```
1      class VolatileCounter {  
2          static volatile int counter = 0;  
3  
4          static void increment() {  
5              counter++;  
6          }  
7      }
```

At first glance, students may think:

- the field is shared
- the field is volatile
- therefore updates are safe

Wrong

volatile does not make counter++ atomic.

Why counter++ Is Not One Action

Conceptually, this statement:

counter++

expands into:

```
1      int temp = counter;  
2      temp = temp + 1;  
3      counter = temp;
```

This is a sequence of steps:

- ① read current value
- ② compute new value
- ③ write new value

Atomicity problem

Another thread can interleave between these steps.

Lost Update Example

```
1      class CounterDemo {  
2          static int counter = 0;  
3  
4          public static void main(String[] args) throws  
5              Exception {  
6              Thread t1 = new Thread(() -> {  
7                  for (int i = 0; i < 100000; i++)  
8                      counter++;  
9              });  
10  
11              Thread t2 = new Thread(() -> {  
12                  for (int i = 0; i < 100000; i++)  
13                      counter++;  
14              });  
15  
16              t1.start();  
17              t2.start();  
18          }  
19      }
```


What Result Do We Expect?

If both threads increment 100000 times, we expect:

200000

But often the observed result is smaller.

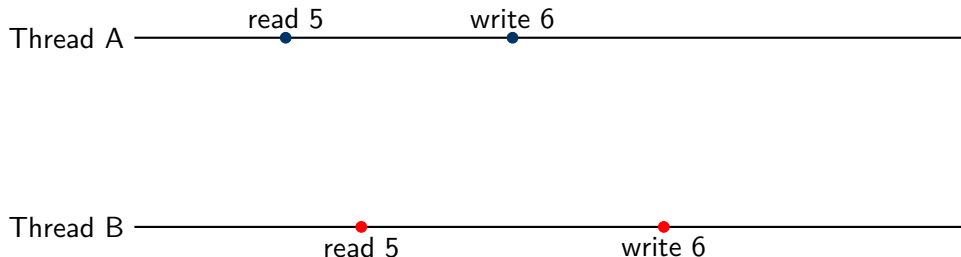
Why?

Because two increments can overlap and one update can overwrite another.

This is called a lost update

One thread's work is effectively erased by another thread's write.

Lost Update Timeline



What happened?

Both threads read the same old value 5. Both compute 6. Both write 6. One increment disappears.

Why volatile Still Fails Here

Suppose we change:

```
1      static int counter = 0;
```

to:

```
1      static volatile int counter = 0;
```

Now:

- reads are visible
- writes are visible

But

The full read-modify-write sequence is still not one indivisible operation.

Conclusion

`volatile` fixes communication visibility, not compound update safety

This Is the Boundary of volatile

What volatile solves	seeing updated values, publication ordering
What volatile does not solve	increment races, check-then-act races, multi-step consistency

Best teaching sentence

volatile tells threads to **see**. It does not force updates to happen as **one step**.

Connecting Back to Week 3

In Week 3, we studied:

- `synchronized`
- `ReentrantLock`

These mechanisms solve a different problem:

- mutual exclusion
- protected critical sections
- atomic behavior for grouped operations

Why this matters now

When the problem is:

"Only one thread must perform this update at a time"

then Week 3 tools remain essential.

Week 3 vs Week 4 Tools

Mechanism	Visibility	Ordering	Atomicity
Plain field	No guarantee	No guarantee	No
volatile	Yes	Yes	No
synchronized	Yes	Yes	Yes
ReentrantLock	Yes	Yes	Yes

Interpretation

Different tools solve different concurrency dimensions. They are not interchangeable.

Now the Famous Example: Double-Checked Locking

Why do programmers attempt double-checked locking (DCL)?

Goal:

- avoid constructing an object more than once
- avoid locking after initialization is already done

Typical use case:

- singleton initialization
- lazy creation of expensive shared objects

Basic idea

Check once without locking. If needed, lock and check again before creating.

Broken Double-Checked Locking

```
1      class Singleton {  
2          private static Singleton instance;  
3  
4          static Singleton getInstance() {  
5              if (instance == null) {  
6                  synchronized (Singleton.class) {  
7                      if (instance == null) {  
8                          instance = new Singleton  
9                              ();  
10                     }  
11                 }  
12             }  
13             return instance;  
14         }  
15     }  
16 }
```


Why This Looks Reasonable

Students often see the code and think:

- outer check avoids unnecessary locking
- inner check prevents duplicate creation
- synchronized protects construction
- therefore the code must be safe

But historically this was broken

The issue is not just mutual exclusion. The issue is also **safe publication** and **reordering**.

Object Creation Is Not One Magical Step

This line:

```
instance = new Singleton();
```

is easy to read as a single action, but conceptually it may involve:

- 1 allocate memory
- 2 initialize the object
- 3 assign the reference to `instance`

Danger

If the reference becomes visible before initialization is fully complete, another thread may observe a non-null but incompletely initialized object.

Why Reordering Matters in DCL

A thread could observe:

- `instance != null`
- and therefore skip synchronization
- but still receive an object whose construction is not safely published

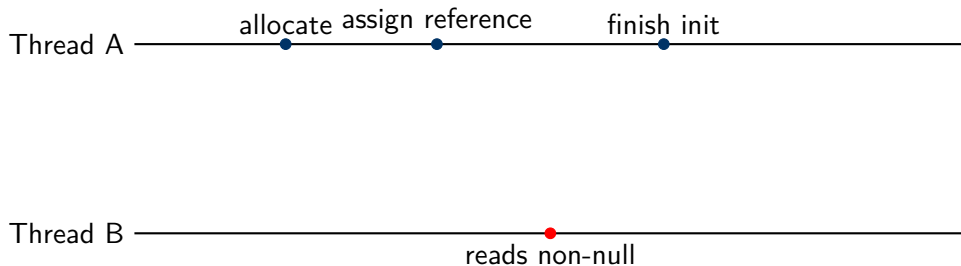
This is subtle

The lock protects mutual exclusion during creation, but the outer unsynchronized read still needs correct visibility and ordering guarantees.

Core lesson

DCL is not just a lock problem. It is also a memory publication problem.

A Timeline View of the DCL Risk



Interpretation

Thread B may observe the shared reference after assignment but before safe initialization is guaranteed to be visible.

Correct DCL with volatile

```
1      class Singleton {
2          private static volatile Singleton instance;
3
4          static Singleton getInstance() {
5              if (instance == null) {
6                  synchronized (Singleton.class) {
7                      if (instance == null) {
8                          instance = new Singleton
9                              ();
10                      }
11                  }
12              }
13              return instance;
14          }
15      }
```

Why volatile Fixes the Publication Side

With:

```
private static volatile Singleton instance;
```

the field now has:

- visibility guarantees
- ordering guarantees relevant to publication

Meaning

Once one thread publishes the initialized instance through the volatile field, later readers of that field see a correctly published object.

Division of roles

- `synchronized` prevents duplicate construction
- `volatile` prevents unsafe publication / harmful reordering

What DCL Teaches Students

DCL is a powerful teaching example because it combines:

- mutual exclusion
- visibility
- ordering
- publication correctness
- performance motivation

Most important lesson

A lock can protect a critical section, but a shared reference still needs correct publication semantics when read outside the lock.

Should Students Use DCL Often?

My opinion: **usually no.**

Even though DCL is correct in modern Java with `volatile`, it is still:

- subtle
- easy to misunderstand
- often less clear than safer alternatives

Teach DCL for understanding

Teach it because it reveals deep concurrency ideas, not because it should be the default design pattern everywhere.

Safer Alternative: Initialization-on-Demand Holder

```
1      class Singleton {  
2          private Singleton() {}  
3  
4          private static class Holder {  
5              static final Singleton INSTANCE = new  
6                  Singleton();  
7          }  
8  
9          static Singleton getInstance() {  
10             return Holder.INSTANCE;  
11         }  
12     }
```

Why this is attractive

It is simpler, cleaner, and avoids many DCL subtleties.

Another Alternative: Eager Initialization

```
1      class Singleton {  
2          private static final Singleton INSTANCE = new  
3              Singleton();  
4  
5          private Singleton() {}  
6  
7          static Singleton getInstance() {  
8              return INSTANCE;  
9          }  
10     }
```

Use when appropriate

If lazy creation is not necessary, eager initialization is often the cleanest option.

Part 3 Comparison Summary

Visibility problem

one thread may not see another thread's write

Atomicity problem

a multi-step update can interleave with another thread

volatile helps with

visibility and ordering

volatile does not help with

compound update atomicity

DCL needs

both exclusion and safe publication

Common Misconceptions to Kill Here

Students should **not** leave this lecture believing:

- “If a field is volatile, increment is safe”
- “If there is a synchronized block somewhere, publication is automatically safe everywhere”
- “Object construction is one indivisible machine step”
- “DCL is just a fancy lock optimization”

Replace with

Correct concurrent design must separately reason about visibility, ordering, atomicity, and publication.

Lab Preparation for Part 3

Lab 4.5 — Lost Update Counter

Students compare:

- `plain int`
- `volatile int`
- synchronized counter from Week 3

and explain the differences.

Lab 4.6 — Broken DCL Analysis

Students inspect DCL and explain why it is not only a locking issue.

Lab 4.7 — Fix DCL

Students add `volatile` and explain exactly what it contributes.

Discussion Questions

- 1 Why does `volatile` fail for `counter++`?
- 2 What exactly is lost in a lost-update race?
- 3 Why are Week 3 locks still relevant in Week 4?
- 4 Why is DCL not just about preventing duplicate construction?
- 5 What role does `volatile` play in correct DCL?
- 6 Why might a simpler singleton pattern be preferable?

Takeaway

Final takeaway for Part 3

A concurrent program can have correct visibility and still be wrong because visibility is not the same as atomicity.

And

Double-checked locking teaches that concurrency correctness is also about safe publication, not only critical-section protection.

Bridge to Part 4

Now that we know `volatile` is not enough for atomic increments, the next question becomes:

How can we perform atomic updates without using a traditional lock?

Questions?